Today
* Arrays
* sizeof();
* Pointer arithmitic
* P++
* GOT
* Ptr vs. Array
* Heap allocations

* Arrays
* arrays do not throw an exception when you go out of bounds.
* arrays take up a contigues space in memory

Size of()

```
Printf("%d", sizeof(a[0])); //4   bytes
sizeof(a) // 40
Size of(int); //4
size of(&a[0]); //8
```

sizeof() takes into the account of the mem adr/pointer = 8 bytes

you can evaluate types to get their size

foo()

```
int a[4]={100, 101, 102, 103};
int *P=&a[0];
```

taking the address of a single element

Changes i?

```
for(int i=0; i<4; i++)
{
  Printf("%d", a[i]);
  Printf("%d", *(P+i));
  Printf("%p", P);
```

iterator → Pointer! → equivalent

in general all Postfix operators have higher precedance than prefix operators

a: | 100 | 101 | 102 | 103 | at 2000, 2004, 2008, 2012

P+t;
Printf("%p", P+t);

P: 2000

Does not change P

- adding to a pointer, you can add a number to a pointers

P+1; means: when you add 1 to a pointer, find the underlying data type, get the size of that type, & add it to the Pointer (the next byte)

- C language allows Pointing 1 past the last element, you can't read it though

this will be on the exam

Grand Unified Theory of Pointers & arrays

```
int a[4]={100};
int *P=&a[0]; or int *P=a;
```

I) a[i] <=> *(P+i)

II) a ⟹ &a[0] (in most cases) exceptions:
sizeof(a)//4
sizeof(&a[0])//8

III) a[i] <=> *(P+i) <=> *(a+i)
a[i] <=> *(a+i)
P[i] <=> (&a[0])[i] = a[i]
P[i] <=> *(P+i)

II. final:

u[i] <=> *(u+i)

array or Pointer

a[2] 2[a] //this will compile by the GOT III.
<=> *(2+a)
it is dumb.
<=> *(a+2) <=> a[2]

Ptr vs arrays
① sizeof(a) vs sizeof(P)
16 8

② P++ a++

③ int f(int a[10]){ ..... } doesn't let you Pass an array in C.
will be ignored
int f(int *a)

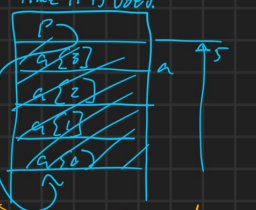
Heap allocations

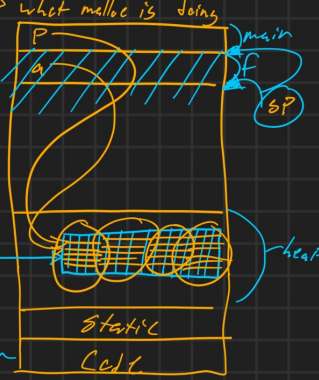
```
int *fC){
  int a[4]={100};
  int *a=malloc(sizeof(int)*4);
  return a;
}
int main(){
  int *P;
  P= fc);
  Printf("%d", P[2]);
  free(P);
}
```

explain this later today do not just will Compile but doesn't work

- a is a stack variable & will become by the time it is used.

we do not own this region of mem





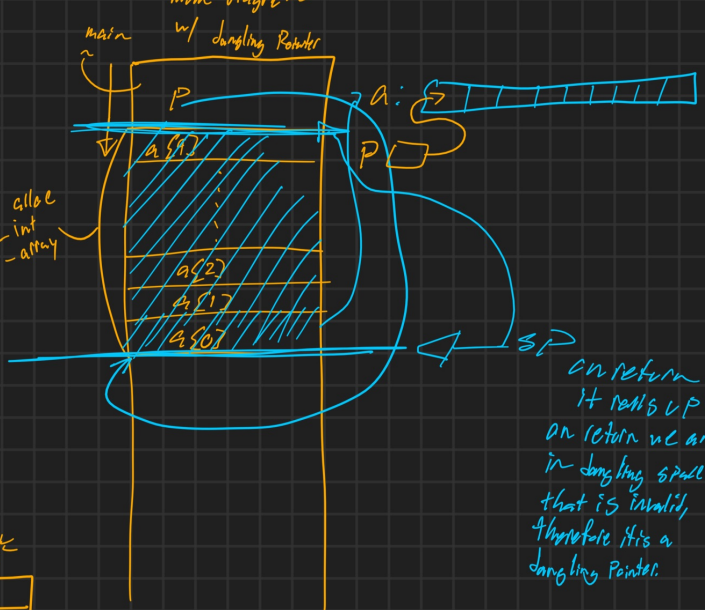
- when f is freed sp rolls back up
- the address will go away and zero and gets assigned to P
- malloc creates the heap and allocates 16 bytes there.
- the function returns the Pointer as *void
- only way to free up the heap is by calling free();

Heap allocation Review & Valgrind:

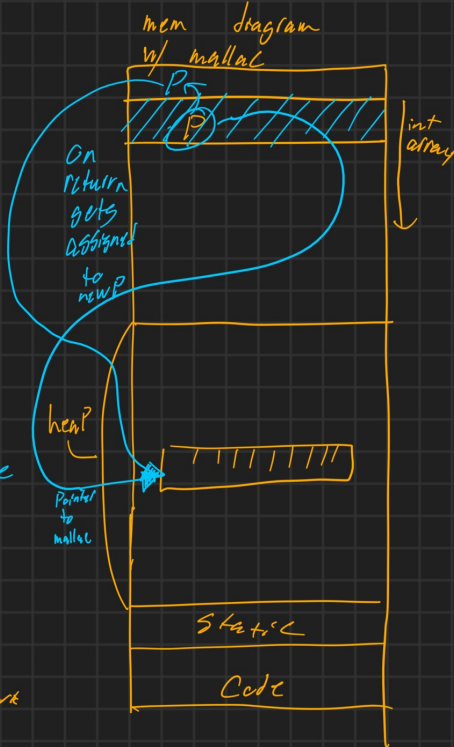
Dangling Pointer:

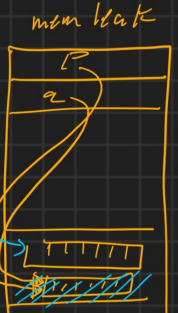
```
int *alloc_int_array(void){
  // Dangling Pointer
  int a[10];
  return a;
  // Allocating mem on heap
  int *P= malloc(sizeof(int)*10);
  return P;
}
void f(int *P, int x){
  *(P+5) = x;
}
int main(){
  int *P = alloc_int_array();
  //f(P, 500);
  Printf("%d\n", P[5]);
  P=NULL;
  P=alloc_int_array();
  f(P, 501);
  Printf("%d\n", P[5]);
```

lost this mem address. memory leak



on return it rolls up on return we are in dangling space that is invalid, therefore it is a dangling Pointer.





- if we instead declare a static var, it would work
- but malloc can allocate & deallocate in the memory.
- so memory footprint would be much lower.
- you can use static if you know the size & want the array to persist throughout the program. ← static vars can't be computed
- locally variables inside function if you just need it in the func.
- Persistance: global local all var

To detect mem leaks: Valgrind

```
valgrind --leak-check=full ./a.out
```

- runs and checks for memory leaks
- must use valgrind to detect invalid access & memory leaks etc.